

◆ HASHING – Concept + Practical Understanding

📖 1 What is Hashing?

Hashing is a technique to **store and retrieve data efficiently** in constant time **$O(1)$** on average.

It converts a **key** into a unique **index (hash value)** of a table (called a **hash table**) using a **hash function**.

⚙️ 2 How Hashing Works

Step

Description

Step 1 Take an input key (e.g., roll number = 101).

Step 2 Apply a **hash function** $\rightarrow \text{index} = \text{key} \% \text{table_size}$.

Step 3 Store the key (and its value) at that index.

Step 4 Retrieve it later using the same hash function.

📁 Example:

Suppose we have 10 slots (0–9) and we want to store these roll numbers:

101, 102, 107, 117

👉 Hash function: $h(\text{key}) = \text{key} \% 10$

Key	$h(\text{key}) = \text{key} \% 10$	Index
101	1	1
102	2	2
107	7	7
117	7	7 $\leftarrow$ ⚠️ Collision!

Both **107** and **117** map to the same index (7).
This situation is called a **collision**.

3 What is a Collision?

A **collision** occurs when **two or more keys hash to the same index** in the hash table.

Since we cannot store both keys in one cell, we need **Collision Resolution Techniques**.

4 Collision Resolution Techniques

There are **two main types**:

➤ (A) Open Hashing (Separate Chaining)

- Each index of the hash table stores a **linked list** (or chain).
- If multiple keys hash to the same index, they are **added to the linked list** at that index.

Advantages

- Simple and flexible.
- Table never gets “full”.

Disadvantages

- Extra memory for pointers.
 - Slower in worst case if too many collisions.
-

Java Example: Separate Chaining

```
import java.util.*;

class HashTableChaining {
    private int size;
    private LinkedList<Integer>[] table;

    public HashTableChaining(int size) {
        this.size = size;
        table = new LinkedList[size];
        for (int i = 0; i < size; i++)
            table[i] = new LinkedList<>();
    }

    private int hash(int key) {
        return key % size;
    }

    public void insert(int key) {
```

```

        int index = hash(key);
        table[index].add(key);
    }

    public void display() {
        for (int i = 0; i < size; i++) {
            System.out.print(i + ": ");
            for (int val : table[i]) {
                System.out.print(val + " -> ");
            }
            System.out.println("null");
        }
    }

    public static void main(String[] args) {
        HashTableChaining ht = new HashTableChaining(10);
        ht.insert(101);
        ht.insert(102);
        ht.insert(107);
        ht.insert(117);
        ht.display();
    }
}

```

Output

```

0: null
1: 101 -> null
2: 102 -> null
3: null
4: null
5: null
6: null
7: 107 -> 117 -> null
8: null
9: null

```

► (B) Closed Hashing (Open Addressing)

In this method, **all keys are stored inside the hash table itself** — no linked lists.

If a collision occurs, we **probe** (search) for another empty slot.

Types:

1. **Linear Probing**
 2. **Quadratic Probing**
 3. **Double Hashing**
-

1 Linear Probing

If a cell is occupied, check the **next cell** (index + 1, index + 2, ...).

 *Formula:*

$h'(key, i) = (h(key) + i) \% table_size$

 *Example in Java*

```
class LinearProbing {
    int[] table;
    int size;

    LinearProbing(int size) {
        this.size = size;
        table = new int[size];
        Arrays.fill(table, -1);
    }

    int hash(int key) {
        return key % size;
    }

    void insert(int key) {
        int index = hash(key);
        int i = 0;
        while (table[(index + i) % size] != -1) {
            i++;
        }
        table[(index + i) % size] = key;
    }

    void display() {
        for (int i = 0; i < size; i++) {
            System.out.println(i + ": " + (table[i] == -1 ? "null" :
table[i]));
        }
    }

    public static void main(String[] args) {
        LinearProbing lp = new LinearProbing(10);
        lp.insert(101);
        lp.insert(102);
        lp.insert(107);
        lp.insert(117);
        lp.display();
    }
}
```

 *Output*

```
0: null
1: 101
2: 102
3: 107
4: 117
5: null
6: null
```

```
7: null
8: null
9: null
```

2 Quadratic Probing

Instead of moving one step at a time, we move in **squares**: $1^2, 2^2, 3^2, \dots$

 *Formula:*

```
h'(key, i) = (h(key) + i*i) % table_size
```

✓ Reduces **clustering** compared to linear probing.

3 Double Hashing

Uses **two hash functions** to determine the probe step.

 *Formula:*

```
h'(key, i) = (h1(key) + i * h2(key)) % table_size
```

✓ Minimizes clustering

✓ Best among open addressing methods

Example:

```
h1(key) = key % size
h2(key) = 7 - (key % 7)
```

5 Comparison Table

Technique	Memory Use	Clustering	Efficiency	Simplicity
Separate Chaining	High	Low	Good	Easy
Linear Probing	Low	High	Moderate	Easy
Quadratic Probing	Low	Medium	Better	Moderate
Double Hashing	Low	Low	Best	Complex

6 Real-Life Analogy

Imagine a **parking lot** (hash table) with slots labeled 0–9. Each car's number plate is the **key**.

- **Hash function** decides where to park.
 - If the slot is full (collision):
 - **Linear Probing**: check next slot.
 - **Quadratic Probing**: check 1^2 , 2^2 , 3^2 slots away.
 - **Chaining**: park behind the first car in a small line.
-

7 Summary for Students

Term	Meaning
Hashing	Fast data access technique
Hash Function	Converts key $\rightarrow$ index
Collision	Two keys get same index
Separate Chaining	Store in linked list
Open Addressing	Store in next empty slot
Load Factor	No. of elements / Table size (keep < 0.7 for performance)

Introduction to Binary Tree

Binary Tree is a non-linear and hierarchical data structure where each node has at most two children referred to as the left child and the right child. The topmost node in a binary tree is called the root, and the bottom-most nodes(having no children) are called leaves.

Representation of Binary Tree

Each node in a Binary Tree has three parts:

- **Data**
- **Pointer to the left child**
- **Pointer to the right child**

Terminologies in Binary Tree

- **Parent Node:** A node that is the direct ancestor of a node(its child node).
- **Child Node:** A node that is the direct descendant of another node (its parent).
- **Ancestors of a node:** All nodes on the path from the root to that node (including the node itself).
- **Descendants of a node:** All nodes that lie in the subtree rooted at that node (including the node itself).
- **Subtree of a node:** A tree consisting of that node as root and all its descendants.
- **Edge:** The link/connection between a parent node and its child node.
- **Path in a binary tree:** A sequence of nodes connected by edges from one node to another.
- **Leaf Node:** A node that does not have any children or both children are null.
- **Internal Node:** A node that has at least one child.
- **Depth/Level of a Node:** The number of edges in the path from root to that node. The depth/level of the root node is zero.
- **Height of a Binary Tree:** The number of edges on the longest path from root to a leaf.

```
// Node structure
class Node {
    int data;
    Node left, right;
    Node(int d) {
        data = d;
        left = null;
        right = null;
    }
}

class GfG {
    public static void main(String[] args) {
        // Initialize and allocate memory for tree nodes
        Node firstNode = new Node(2);
        Node secondNode = new Node(3);
        Node thirdNode = new Node(4);
        Node fourthNode = new Node(5);

        // Connect binary tree nodes
        firstNode.left = secondNode;
        firstNode.right = thirdNode;
        secondNode.left = fourthNode;

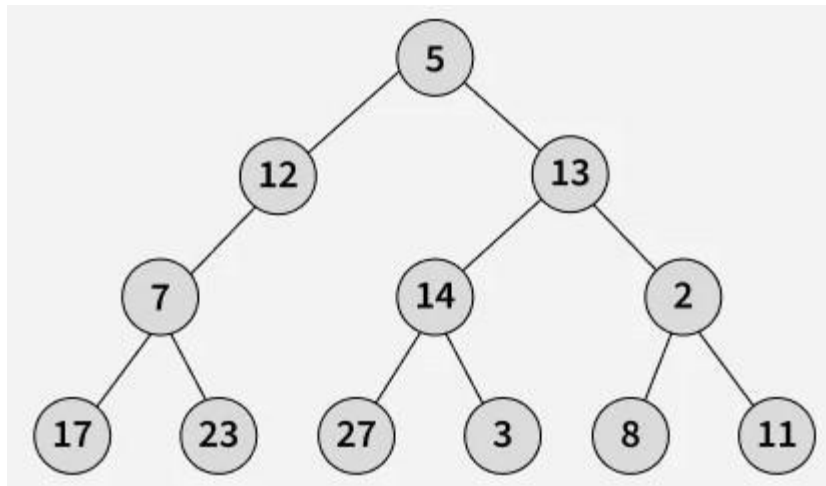
    }
}
```

Level Order Traversal (Breadth First Search) of Binary Tree

Given a **root** of the Binary Tree, find its **Level Order Traversal**.

Level Order Traversal technique is a method to traverse a Tree such that all nodes present in the same level are traversed completely before traversing the next level.

Example:



```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;

class Node {
    int data;
    Node left, right;
    Node(int value)
    {
        data = value;
        left = null;
        right = null;
    }
}
```

```

// Iterative method to perform level order traversal
public class GfG {
    public static ArrayList<ArrayList<Integer>> levelOrder(Node root)
    {
        if (root == null)
            return new ArrayList<>();

        // Create an empty queue for level order traversal
        Queue<Node> q = new LinkedList<>();
        ArrayList<ArrayList<Integer>> res = new ArrayList<>();

        // Enqueue Root
        q.offer(root);
        int currLevel = 0;

        while (!q.isEmpty()) {
            int len = q.size();
            res.add(new ArrayList<>());

            for (int i = 0; i < len; i++) {
                // Add front of queue and remove it from
                // queue
                Node node = q.poll();
                res.get(currLevel).add(node.data);

                // Enqueue left child
                if (node.left != null)
                    q.offer(node.left);

                // Enqueue right child
                if (node.right != null)
                    q.offer(node.right);
            }
            currLevel++;
        }
        return res;
    }

    public static void main(String[] args)
    {
        //          5
        //        / \
        //       12  13
        //      / \  \
        //     7  14  2
        //    / \ / \ / \
        //   17 23 27 3 8 11

        Node root = new Node(5);
        root.left = new Node(12);
        root.right = new Node(13);
    }
}

```

```

    root.left.left = new Node(7);
    root.left.right = new Node(14);

    root.right.right = new Node(2);

    root.left.left.left = new Node(17);
    root.left.left.right = new Node(23);

    root.left.right.left = new Node(27);
    root.left.right.right = new Node(3);

    root.right.right.left = new Node(8);
    root.right.right.right = new Node(11);

    // Perform level order traversal and get the result
    ArrayList<ArrayList<Integer>> res = levelOrder(root);

    for (ArrayList<Integer> level : res) {
        for (int val : level) {
            System.out.print(val + " ");
        }
        System.out.println();
    }
}

```

Insertion in a Binary Tree in level order

```

// Java program to insert element (in level order)
// in Binary Tree
import java.util.LinkedList;
import java.util.Queue;

class Node {
    int data;
    Node left, right;

    Node(int x) {
        data = x;
        left = right = null;
    }
}

class GfG {

    // Function to insert element
    // in binary tree
    static Node InsertNode(Node root, int data) {

        // If the tree is empty, assign new node

```

```

        // address to root
        if (root == null) {
            root = new Node(data);
            return root;
        }

        // Else, do level order traversal until we find an empty
        // place, i.e. either left child or right child of some
        // node is pointing to NULL.
        Queue<Node> q = new LinkedList<>();
        q.add(root);

        while (!q.isEmpty()) {

            // Fron a front element in queue
            Node curr = q.poll();

            // First check left if left is null insert
            // node in left otherwise chaeck for right
            if (curr.left != null)
                q.add(curr.left);
            else {
                curr.left = new Node(data);
                return root;
            }

            if (curr.right != null)
                q.add(curr.right);
            else {
                curr.right = new Node(data);
                return root;
            }
        }
        return root;
    }

    // Inorder traversal of a binary tree
    static void inorder(Node curr) {
        if (curr == null)
            return;
        inorder(curr.left);
        System.out.print(curr.data + " ");
        inorder(curr.right);
    }

    public static void main(String[] args) {

        // Constructing the binary tree
        //          10
        //         /  \
        //        11   9
        //       /    / \

```

```

//      7      15   8
Node root = new Node(10);
root.left = new Node(11);
root.right = new Node(9);
root.left.left = new Node(7);
root.right.left = new Node(15);
root.right.right = new Node(8);

int key = 12;
root = InsertNode(root, key);

// After insertion 12 in binary tree
//           10
//        /   \
//       11    9
//      / \   / \
//     7  12 15  8
inorder(root);
}
}

```

Operations:

Operation	Description
Insert	Add a new node at the first available position (level order).
Delete	Remove a node by value (replace with deepest rightmost node).
Traversal	Visit all nodes in a specific order.

2 Java Implementation

```

import java.util.*;

class BinaryTree {
    // Node structure
    static class Node {
        int data;
        Node left, right;

        Node(int value) {
            data = value;
            left = right = null;
        }
    }

    Node root;

    // Constructor

```

```

BinaryTree() {
    root = null;
}

// -----
// INSERT (Level Order)
// -----
void insert(int key) {
    Node newNode = new Node(key);

    if (root == null) {
        root = newNode;
        return;
    }

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        Node temp = q.poll();

        if (temp.left == null) {
            temp.left = newNode;
            break;
        } else {
            q.add(temp.left);
        }

        if (temp.right == null) {
            temp.right = newNode;
            break;
        } else {
            q.add(temp.right);
        }
    }
}

// -----
// DELETE a node
// -----
void delete(int key) {
    if (root == null)
        return;

    if (root.left == null && root.right == null) {
        if (root.data == key)
            root = null;
        return;
    }

    Queue<Node> q = new LinkedList<>();
    q.add(root);
    Node keyNode = null, temp = null;

    // Find the node to delete and the last node
    while (!q.isEmpty()) {
        temp = q.poll();

        if (temp.data == key)

```

```

        keyNode = temp;

        if (temp.left != null)
            q.add(temp.left);

        if (temp.right != null)
            q.add(temp.right);
    }

    if (keyNode != null) {
        int deepestValue = temp.data;
        deleteDeepest(temp);
        keyNode.data = deepestValue;
    }
}

// Helper: delete deepest node
void deleteDeepest(Node delNode) {
    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        Node temp = q.poll();

        if (temp == delNode) {
            temp = null;
            return;
        }

        if (temp.left != null) {
            if (temp.left == delNode) {
                temp.left = null;
                return;
            } else
                q.add(temp.left);
        }

        if (temp.right != null) {
            if (temp.right == delNode) {
                temp.right = null;
                return;
            } else
                q.add(temp.right);
        }
    }
}

// -----
// TREE TRAVERSALS
// -----
void inorder(Node node) {
    if (node == null)
        return;
    inorder(node.left);
    System.out.print(node.data + " ");
    inorder(node.right);
}

```

```

void preorder(Node node) {
    if (node == null)
        return;
    System.out.print(node.data + " ");
    preorder(node.left);
    preorder(node.right);
}

void postorder(Node node) {
    if (node == null)
        return;
    postorder(node.left);
    postorder(node.right);
    System.out.print(node.data + " ");
}

// -----
// MAIN METHOD
// -----
public static void main(String[] args) {
    BinaryTree bt = new BinaryTree();

    bt.insert(10);
    bt.insert(20);
    bt.insert(30);
    bt.insert(40);
    bt.insert(50);

    System.out.println("Inorder traversal:");
    bt.inorder(bt.root);

    System.out.println("\nPreorder traversal:");
    bt.preorder(bt.root);

    System.out.println("\nPostorder traversal:");
    bt.postorder(bt.root);

    System.out.println("\n\nDeleting node 20...");
    bt.delete(20);

    System.out.println("Inorder after deletion:");
    bt.inorder(bt.root);
}
}

```

3 Output

```

Inorder traversal:
40 20 50 10 30
Preorder traversal:
10 20 40 50 30
Postorder traversal:
40 50 20 30 10

```

```
Deleting node 20...
Inorder after deletion:
40 30 50 10 30
```

4 Explanation

Step	Operation	Description
1	<code>insert(10)</code>	Root = 10
2	<code>insert(20),</code> <code>insert(30)</code>	Added level-wise
3	<code>insert(40),</code> <code>insert(50)</code>	Added to next level
4	<code>delete(20)</code>	Finds node 20 and replaces with deepest node (50), deletes deepest node

5 Traversal Summary

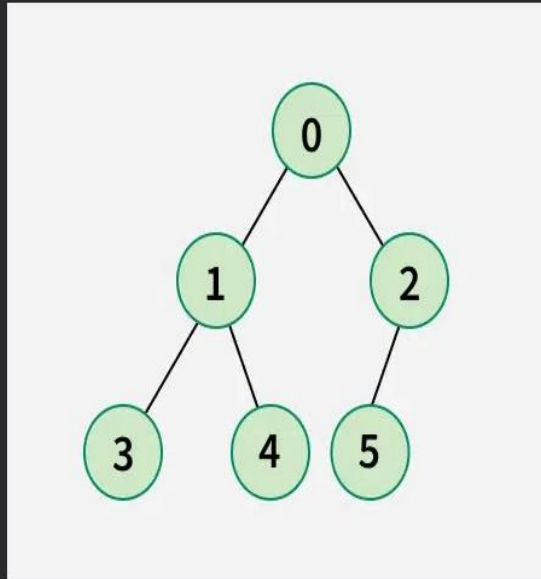
Type	Order	Description
Inorder	Left → Root → Right	Yields sorted order in BST
Preorder	Root → Left → Right	Used to copy tree
Postorder	Left → Right → Root	Used to delete tree

Binary Tree from Inorder and Preorder traversals

Input: `inorder[] = [3, 1, 4, 0, 5, 2], preorder[] = [0, 1, 3, 4, 2, 5]`

Output: `[[0], [1, 2], [3, 4, 5, N]]`

Explanation: The tree will look like:



```
import java.util.*;

// Node Structure
class Node {
    int data;
    Node left, right;
    Node(int x) {
        data = x;
        left = right = null;
    }
}
```

```

class BinaryTreeBuilder {

    static int preIndex = 0; // Global index for preorder array

    // Recursive function to build the binary tree
    static Node buildTreeRecur(Map<Integer, Integer> inorderMap, int[]
preorder, int left, int right) {
        // Base case
        if (left > right) return null;

        // Pick current node from preorder traversal
        int rootVal = preorder[preIndex++];
        Node root = new Node(rootVal);

        // Get inorder index of this node
        int index = inorderMap.get(rootVal);

        // Build left and right subtrees
        root.left = buildTreeRecur(inorderMap, preorder, left, index -
1);
        root.right = buildTreeRecur(inorderMap, preorder, index + 1,
right);

        return root;
    }

    // Function to construct tree from inorder and preorder traversals
    static Node buildTree(int[] inorder, int[] preorder) {
        preIndex = 0; // Reset before building a new tree

        // Create a hashmap for quick lookup of inorder indices
        Map<Integer, Integer> inorderMap = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) {
            inorderMap.put(inorder[i], i);
        }

        return buildTreeRecur(inorderMap, preorder, 0, inorder.length
- 1);
    }

    // Function to calculate height of tree
    static int getHeight(Node root) {
        if (root == null) return -1;
        return Math.max(getHeight(root.left), getHeight(root.right)) +
1;
    }
}

```

```

// Function to print level order traversal including nulls as 'N'
static void levelOrder(Node root) {
    if (root == null) return;

    Queue<List<Object>> queue = new LinkedList<>();
    queue.offer(List.of(root, 0));

    int lastLevel = 0;
    int height = getHeight(root);

    while (!queue.isEmpty()) {
        List<Object> top = queue.poll();
        Node node = (Node) top.get(0);
        int lvl = (int) top.get(1);

        if (lvl > lastLevel) {
            System.out.println();
            lastLevel = lvl;
        }

        if (lvl > height) break;

        System.out.print((node.data == -1 ? "N" : node.data) + "
");

        if (node.data == -1) continue;

        queue.offer(List.of(node.left == null ? new Node(-1) :
node.left, lvl + 1));
        queue.offer(List.of(node.right == null ? new Node(-1) :
node.right, lvl + 1));
    }
}

public static void main(String[] args) {
    int[] inorder = {3, 1, 4, 0, 5, 2};
    int[] preorder = {0, 1, 3, 4, 2, 5};

    Node root = buildTree(inorder, preorder);

    System.out.println("Level Order Traversal (including
nulls):");
    levelOrder(root);
}
}

```

1) Preorder Traversal (recursive)

Problem statement

Given the root of a binary tree, return the preorder traversal of its nodes' values (Root → Left → Right).

Pattern

Depth-First Search (DFS) — **Root, Left, Right**.

Solution trick to remember

Visit current node first, then recurse left, then right. Think: "Speak (root) → explore left → explore right."

Variations

- Return as `vector<int>` (flat).
- Return as `vector<vector<int>>` grouped by level (rare).
- Iterative preorder (stack) — useful when recursion depth is a concern.
- Morris preorder ($O(1)$ extra space).

Clean C++ (recursive)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x): val(x), left(nullptr), right(nullptr) {}
};

// Preorder recursive: Root, Left, Right
void preorderDFS(TreeNode* node, vector<int>& out) {
    if (!node) return;
    out.push_back(node->val);
    preorderDFS(node->left, out);
    preorderDFS(node->right, out);
}

vector<int> preorderTraversal(TreeNode* root) {
    vector<int> res;
    preorderDFS(root, res);
    return res;
}
```

Time complexity: $O(n)$ — visit each node once.

Space complexity: $O(h)$ recursion stack (h = tree height). Worst-case $O(n)$ for skewed tree, average $O(\log n)$ for balanced.

2) Inorder Traversal (recursive)

Problem statement

Given a binary tree root, return the inorder traversal (Left → Root → Right).

Pattern

DFS — **Left, Root, Right**.

Solution trick to remember

Inorder gives sorted order for BSTs. Think: "Explore left → speak (root) → explore right." For BST-related problems, inorder is often the key.

Variations

- Iterative inorder using stack.
- Morris inorder for $O(1)$ extra space.
- Use inorder to validate BST or to get sorted list of values.

Clean C++ (recursive)

```
#include <bits/stdc++.h>
using namespace std;

void inorderDFS(TreeNode* node, vector<int>& out) {
    if (!node) return;
    inorderDFS(node->left, out);
    out.push_back(node->val);
    inorderDFS(node->right, out);
}

vector<int> inorderTraversal(TreeNode* root) {
    vector<int> res;
    inorderDFS(root, res);
    return res;
}
```

Time complexity: $O(n)$.

Space complexity: $O(h)$ recursion stack.

3) Post-order Traversal (recursive)

Problem statement

Given root of binary tree, return postorder traversal (Left → Right → Root).

Pattern

DFS — **Left, Right, Root.**

Solution trick to remember

This is useful when you need child results before processing the parent (e.g., tree deletion, computing subtree values). Mnemonic: "Do children → finish with parent."

Variations

- Iterative postorder using two stacks.
- Iterative postorder using one stack + prev pointer.
- Trick: do modified preorder (Root, Right, Left) then reverse result → gives postorder.

Clean C++ (recursive)

```
#include <bits/stdc++.h>
using namespace std;

void postorderDFS(TreeNode* node, vector<int>& out) {
    if (!node) return;
    postorderDFS(node->left, out);
    postorderDFS(node->right, out);
    out.push_back(node->val);
}

vector<int> postorderTraversal(TreeNode* root) {
    vector<int> res;
    postorderDFS(root, res);
    return res;
}
```

Time complexity: $O(n)$.

Space complexity: $O(h)$ recursion stack.

4) Level Order Traversal / Spiral (Zigzag)

Level Order Traversal

Problem statement

- **Level Order:** Return nodes level by level (top → bottom), each level left → right.
- **Spiral (Zigzag) Level Order:** Alternate direction per level (level0 left→right, level1 right→left, ...).

Pattern

Breadth-First Search (BFS) using a queue for level order. For zigzag, either:

- use a deque and toggle insertion direction, or
- use two stacks (one for current level, one for next) and alternate push order.

Solution trick to remember

Level order = BFS. Use `queue` and loop by `q.size()` to capture one level. For zigzag: toggle direction boolean; when direction is right-to-left push values in reverse order (or push to deque accordingly).

Variations

- Return `vector<vector<int>>` (levels separated).
- Return flattened vector.
- Spiral order using two stacks (classic).
- Use deque to avoid reversing levels.

Clean C++ (level-order as `vector<vector<int>>`)

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        level.reserve(sz);
        for (int i = 0; i < sz; ++i) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(move(level));
    }
    return res;
}
```

Clean C++ (spiral/zigzag — two stacks approach, returns flattened vector)

```
#include <bits/stdc++.h>
using namespace std;

vector<int> spiralOrder(TreeNode* root) {
    vector<int> res;
    if (!root) return res;
    stack<TreeNode*> cur, next;
    cur.push(root);
    bool leftToRight = true;
    while (!cur.empty()) {
        if (leftToRight) {
            while (!cur.empty()) {
                res.push_back(cur.top()->val);
                cur.pop();
            }
            while (!next.empty()) {
                cur.push(next.top());
                next.pop();
            }
        } else {
            while (!cur.empty()) {
                res.push_back(cur.top()->val);
                cur.pop();
            }
            while (!next.empty()) {
                cur.push(next.top());
                next.pop();
            }
        }
        leftToRight = !leftToRight;
    }
    return res;
}
```

```

while (!cur.empty()) {
    TreeNode* node = cur.top(); cur.pop();
    res.push_back(node->val);

    if (leftToRight) {
        if (node->left) next.push(node->left);
        if (node->right) next.push(node->right);
    } else {
        if (node->right) next.push(node->right);
        if (node->left) next.push(node->left);
    }

    if (cur.empty()) {
        swap(cur, next);
        leftToRight = !leftToRight;
    }
}
return res;
}

```

Time complexity: $O(n)$ for both.

Space complexity: $O(w)$ where w = max width of tree (queue or stacks) — worst-case $O(n)$.

5) Iterative Preorder Traversal

Problem statement

Return preorder traversal ($\text{Root} \rightarrow \text{Left} \rightarrow \text{Right}$) **iteratively** (no recursion).

Pattern

DFS using an explicit stack. **Push right child first, then left child** — so left is processed before right.

Solution trick to remember

Stack processes LIFO. To simulate **Root-L-R**, push right first, then left. This guarantees left is popped before right.

Variations

- Iterative using stack of pairs (node,state) to simulate all three traversals in one pass.
- Morris preorder for $O(1)$ space (threading technique).

Clean C++ (iterative)

```

#include <bits/stdc++.h>
using namespace std;

vector<int> preorderIterative(TreeNode* root) {
    vector<int> res;

```

```

if (!root) return res;
stack<TreeNode*> st;
st.push(root);
while (!st.empty()) {
    TreeNode* node = st.top(); st.pop();
    res.push_back(node->val);
    // push right first so left gets processed next
    if (node->right) st.push(node->right);
    if (node->left) st.push(node->left);
}
return res;
}

```

Time complexity: $O(n)$.

Space complexity: $O(h)$ stack (worst-case $O(n)$).

Quick summary & memory hooks

- **Preorder:** Root first $\rightarrow$ "Speak first". Recursive or push right then left on stack.
- **Inorder:** Left $\rightarrow$ Root $\rightarrow$ Right $\rightarrow$ "Sorted for BST". Morris if $O(1)$ needed.
- **Postorder:** Children before parent $\rightarrow$ useful for aggregations/deletions. Use reversed modified-preorder trick to get iterative postorder quickly.
- **Level-order:** BFS via queue; use `q.size()` to separate levels.
- **Spiral:** Toggle direction — two stacks or deque.
- **Iterative Preorder:** Stack, push right then left.

1) Iterative Inorder Traversal

Problem statement

Given a binary tree root, return inorder traversal (Left $\rightarrow$ Root $\rightarrow$ Right) **without recursion**.

Pattern

DFS using **explicit stack**.

Solution trick to remember

- **Go as left as possible, push nodes on stack.**
- **When left is null, pop stack $\rightarrow$ visit node $\rightarrow$ move to right.**
- **Repeat until stack empty & current null.**

Variations

- Return `vector<int>` (flat).
- Can also be modified to get preorder/postorder in one pass using state pairs.

C++ code

```
vector<int> inorderIterative(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    TreeNode* curr = root;

    while (curr || !st.empty()) {
        while (curr) {
            st.push(curr);
            curr = curr->left;
        }
        curr = st.top(); st.pop();
        res.push_back(curr->val);
        curr = curr->right;
    }
    return res;
}
```

Time: O(n), **Space:** O(h)

2) Post-order Traversal using 2 Stacks

Problem statement

Return postorder traversal (Left → Right → Root) iteratively using **two stacks**.

Pattern

DFS simulated with **two stacks**.

Solution trick to remember

- Push nodes to first stack (st1).
- Pop from st1 → push children to st1 (left first, right next) → push node to st2.
- Pop st2 → gives postorder

Same as preorder iterative diff is first push left then child in st1, now ans stored in reverse order so pop it from st2

C++ code

```
vector<int> postorderTwoStacks(TreeNode* root) {
    if (!root) return {};
    stack<TreeNode*> st1, st2;
    st1.push(root);
    while (!st1.empty()) {
        TreeNode* node = st1.top(); st1.pop();
        st2.push(node);
        if (node->left) st1.push(node->left);
    }
}
```

```

        if (node->right) st1.push(node->right);
    }
    vector<int> res;
    while (!st2.empty()) {
        res.push_back(st2.top()->val);
        st2.pop();
    }
    return res;
}

```

Time: O(n), **Space:** O(n)

OR

```

vector<int> postorderTraversal(TreeNode* root) {
    if (!root) return {};
    stack<TreeNode*> st;
    vector<int> res;

    st.push(root);
    while (!st.empty()) {
        TreeNode* node = st.top(); st.pop();
        res.push_back(node->val);

        if (node->left) st.push(node->left);
        if (node->right) st.push(node->right);
    }
    reverse(res.begin(), res.end());
    return res;
}

```

3) Post-order Traversal using 1 Stack

Problem statement

Return postorder traversal ($L \rightarrow R \rightarrow \text{Root}$) **iteratively using one stack**.

Pattern

DFS with **stack + previous pointer**.

Solution trick to remember

- Track previously visited node (`prev`).
- Check if moving down, left, or right.
- Only visit node if children are done.



Problem: Postorder Traversal (Left $\rightarrow$ Right $\rightarrow$ Root) using 1 Stack

First, Recall the Rule

Postorder means:

👉 Go to **Left Child** → then **Right Child** → then **Process the Node** (Root last).

Why It's Hard?

Because we can't directly "see" whether a node's right child is already processed or not. That's why we need **extra memory (prev pointer)** to remember:

➡ "Have I already visited the right child?"

Visual Trick (Story mode)

Imagine a **school trip** 🚌 to a Tree 🌳.

- You (stack) keep track of kids (nodes).
- Rule:
 1. Always go **left first**.
 2. If no left → check right.
 3. If both done → say **bye** to that kid (process it and remove from stack).

And you keep a "**previous visited kid**" **notebook** (prev) 📖 to not repeat right child.

Algorithm in Steps (like playing a game 🎮)

1. Start at root.
 2. Keep going left, push every node in stack.
 3. When no left:
 - Look at top of stack (don't pop yet).
 - If **right child exists** and is not the one we just processed → Go right.
 - Else → Process the node (add to result + pop).
 - Mark that node as `prev`.
-

How to Remember Forever (Shortcut Trick)

👉 **Think of it like: "Don't leave your house (node) until both left and right neighbors are visited."**

- Left visited? ☒ Check right.
- Right visited? ☒ Say bye (process node).

That's it 🔥

Code (Clean + Rememberable)

```
vector<int> postorderOneStack(TreeNode* root) {
    vector<int> res;
    if (!root) return res;

    stack<TreeNode*> st;
    TreeNode* curr = root;
    TreeNode* prev = nullptr;

    while (!st.empty() || curr) {
        if (curr) {
            st.push(curr);          // Step 1: Go left
            curr = curr->left;
        } else {
            TreeNode* node = st.top();
            // Step 2: If right child exists and not yet visited → go right
            if (node->right && node->right != prev) {
                curr = node->right;
            } else {
                // Step 3: Both children done → process node
                res.push_back(node->val);
                prev = node;
                st.pop();
            }
        }
    }
    return res;
}
```

Memory Peg

- **2 stacks** → easy (reverse trick).
- **1 stack** → use **prev** like a "visited notebook".

👉 Just say to yourself:

💡 "I will only leave a node when both kids are visited."

Time: $O(n)$, **Space:** $O(h)$

Summary trick: for iterative traversals

While loop condition:

Preorder : !st.empty()

Inorder : curr || !st.empty()

Postorder: 2 stack : !st.empty()

1 stack : curr || !st.empty()

4) Preorder + Inorder + Postorder in One Traversal

Problem statement

Return **preorder, inorder, and postorder traversals** in one DFS traversal.

Pattern

DFS using **stack of pair(node, state)**.

- State 1 $\rightarrow$ preorder, move left
- State 2 $\rightarrow$ inorder, move right
- State 3 $\rightarrow$ postorder, pop

Solution trick to remember

- Use integer state to simulate traversal phases.

C++ code

```
struct Pair {
    TreeNode* node;
    int state;
    Pair(TreeNode* n, int s): node(n), state(s) {}
};

void allTraversals(TreeNode* root, vector<int>& pre, vector<int>& in,
vector<int>& post) {
    if (!root) return;
    stack<Pair> st;
    st.push(Pair(root,1));
```

```

while(!st.empty()) {
    Pair &p = st.top();
    if (p.state == 1) {
        pre.push_back(p.node->val);
        p.state++;
        if (p.node->left) st.push(Pair(p.node->left,1));
    } else if (p.state == 2) {
        in.push_back(p.node->val);
        p.state++;
        if (p.node->right) st.push(Pair(p.node->right,1));
    } else {
        post.push_back(p.node->val);
        st.pop();
    }
}
}

```

Time: $O(n)$, **Space:** $O(h)$

5) Zig-Zag Traversal (Spiral Level Order)

Problem statement

Return **level order** but alternate direction per level.

Pattern

BFS using **two stacks** or **queue + reverse vector**.

Solution trick to remember

- Toggle direction flag per level.
- If using queue $\rightarrow$ reverse vector every other level.
- If using two stacks $\rightarrow$ push left/right in opposite order per level.

C++ code (queue + vector)

```

vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    bool leftToRight = true;

    while(!q.empty()) {
        int sz = q.size();
        vector<int> level(sz);
        for (int i = 0; i < sz; ++i) {
            TreeNode* node = q.front(); q.pop();
            int idx = leftToRight ? i : sz - 1 - i;

```

```

        level[idx] = node->val;
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    leftToRight = !leftToRight;
    res.push_back(level);
}
return res;
}

```

Time: $O(n)$, **Space:** $O(w)$ (width of tree)

6) Morris Preorder Traversal

Problem statement

Return preorder traversal **without recursion and without stack** ($O(1)$ extra space).

Pattern

Threaded binary tree — use rightmost node of left subtree to point back to root.

Solution trick to remember

- If left child exists → find predecessor → make thread → go left.
- Else → visit → go right.
- Remove thread after visiting left subtree.



Key Idea of Morris Traversal

Normally, recursion/stack helps us "come back" to the parent after visiting left subtree.

But Morris says:

🔑 Let's **create a temporary link from the rightmost node of the left subtree (predecessor) back to the current node.**

This way we don't forget where to return.

Think like tying a rope:

- "I'll visit left side → tie a rope back to me → when I'm done, I'll come back by rope."
-



Breaking into Sub-Problems

1. Check if current node has a left child

- If **no left child**: Just process (print/push value) and go right.
 - If **yes**: Find the "predecessor".
-

2. Find predecessor

Predecessor = rightmost node in left subtree.

Why rightmost?

Because after finishing left subtree, we must go **back up** to current.

3. Two cases with predecessor

- If predecessor's right pointer is **null**:
 - 👉 First time visiting this subtree.
 - Process current node (because preorder = visit before going left).
 - Create a thread (link) from predecessor → current.
 - Move left.
 - If predecessor's right pointer is **current**:
 - 👉 Second time we are back.
 - Remove the thread.
 - Move right.

Trick to Remember

👉 Think of Morris as **two visits** to a node:

1. First time → **process & create rope** (go left).
2. Second time → **remove rope** (go right).

🔑 Preorder = process in **first visit**.
(For inorder, we process in second visit!)

Clean Code (C++)

```

vector<int> morrisPreorder(TreeNode* root) {
    vector<int> res;
    TreeNode* curr = root;

    while (curr) {
        if (!curr->left) {
            res.push_back(curr->val);    // process root
            curr = curr->right;          // move right
        } else {
            TreeNode* pred = curr->left;
            while (pred->right && pred->right != curr)
                pred = pred->right;

            if (!pred->right) {
                res.push_back(curr->val); // process root (first time)
                pred->right = curr;      // create thread
                curr = curr->left;        // move left
            } else {
                pred->right = nullptr;   // remove thread
                curr = curr->right;       // move right
            }
        }
    }
    return res;
}

```



Complexity

- **Time:** $O(n)$ (each edge used at most twice).
 - **Space:** $O(1)$ (no recursion, no stack).
-



How to Remember Forever

- Morris = **Threading Technique**.
 - Two visits to a node:
 - 1st visit: Create link, process root (for preorder).
 - 2nd visit: Remove link.
 - **Inorder vs Preorder vs Postorder Morris** only differ in **when you process node**.
-



Variations / Questions You Can Solve

1. **Morris Inorder Traversal** (process in 2nd visit).

2. **Morris Preorder Traversal** (process in 1st visit).
3. **Morris Postorder Traversal** (bit tricky → requires reversing edge traversal).
4. Convert **recursive tree problems** into **O(1) space** versions:
 - Kth smallest/largest (using Morris Inorder).
 - Checking BST validity (inorder Morris).
 - Serialize/deserialize without stack.

Time: O(n), **Space:** O(1)

7) Morris Inorder Traversal

Problem statement

Return inorder traversal **without recursion and without stack**.

Pattern

Same threaded tree idea as Morris preorder.

Solution trick to remember

- Visit node **after finishing left subtree** (opposite of preorder).

C++ code

```
vector<int> morrisInorder(TreeNode* root) {
    vector<int> res;
    TreeNode* curr = root;
    while (curr) {
        if (!curr->left) {
            res.push_back(curr->val);
            curr = curr->right;
        } else {
            TreeNode* pred = curr->left;
            while (pred->right && pred->right != curr) pred = pred->right;
            if (!pred->right) {
                pred->right = curr;
                curr = curr->left;
            } else {
                pred->right = nullptr;
                res.push_back(curr->val);
                curr = curr->right;
            }
        }
    }
    return res;
}
```

Time: O(n), **Space:** O(1)

✓ Memory hooks for these 7 advanced traversals

Traversal	Key Trick
Iterative Inorder	Go left as much as possible → stack
Postorder 2 stacks	Push node to st2 after pushing children to st1
Postorder 1 stack	Use <code>prev</code> pointer to track last visited
All-in-one	Use <code>state</code> to track traversal phase
Zig-Zag	Toggle level direction; insert in index <code>i</code> or <code>sz-1-i</code>
Morris Preorder	Thread → visit before left
Morris Inorder	Thread → visit after left

1) Height of a Binary Tree

Problem statement

Given a binary tree, find its height (maximum depth from root to leaf).

Pattern

DFS recursion — **post-order traversal**.

Solution trick to remember

- Height = max(left subtree height, right subtree height) + 1.
- Always compute child heights first → then parent.

Variations

- Minimum depth
- Height of N-ary tree
- Height during level-order (BFS)

C++ code

```
int height(TreeNode* root) {  
    if (!root) return 0;  
    return 1 + max(height(root->left), height(root->right));  
}
```

Time: O(n)

Space: O(h) recursion stack

2) Check if Binary Tree is Height-Balanced

Problem statement

Check if for every node, height difference between left & right ≤ 1 .

Pattern

DFS post-order, calculate heights bottom-up.

Solution trick to remember

- Return -1 if subtree is unbalanced.
- Height = $\max(\text{left}, \text{right}) + 1$ if balanced; propagate -1 if any child unbalanced.

Variations

- Check AVL tree property
- Balanced at K-level

C++ code

```
int checkHeight(TreeNode* root) {
    if (!root) return 0;
    int lh = checkHeight(root->left);
    if (lh == -1) return -1;
    int rh = checkHeight(root->right);
    if (rh == -1) return -1;
    if (abs(lh - rh) > 1) return -1;
    return 1 + max(lh, rh);
}

bool isBalanced(TreeNode* root) {
    return checkHeight(root) != -1;
}
```

Time: $O(n)$

Space: $O(h)$

3) Diameter of Binary Tree

Problem statement

Diameter = length of longest path between any two nodes (number of edges or nodes).

Pattern

Post-order DFS, calculate height at each node and update max diameter.

Solution trick to remember

- For node: diameter through node = left height + right height
- Keep global variable `res` to track maximum

Variations

- Edge-based or node-based diameter
- Diameter of N-ary tree

C++ code

```
int diameterDFS(TreeNode* root, int &res) {
    if (!root) return 0;
    int lh = diameterDFS(root->left, res);
    int rh = diameterDFS(root->right, res);
    res = max(res, lh + rh);
    return 1 + max(lh, rh);
}

int diameterOfBinaryTree(TreeNode* root) {
    int res = 0;
    diameterDFS(root, res);
    return res;
}
```

Time: $O(n)$

Space: $O(h)$

4) Maximum Path Sum

Problem statement

Find the maximum sum of values along any path (path can start & end at any node).

Pattern

DFS post-order, compute max path sum for each subtree.

Solution trick to remember

- For node: max single path = `node.val + max(left, right, 0)`
- Update global max = `max(global, left+right+node.val)`

Variations

- Path must start from root
- Path must end at leaf

C++ code

```
int maxPathDFS(TreeNode* root, int &res) {
    if (!root) return 0;
    int left = max(0, maxPathDFS(root->left, res));
    int right = max(0, maxPathDFS(root->right, res));
    res = max(res, left + right + root->val);
    return root->val + max(left, right);
}

int maxPathSum(TreeNode* root) {
    int res = INT_MIN;
    maxPathDFS(root, res);
    return res;
}
```

Time: $O(n)$

Space: $O(h)$

5) Maximum Width of a Binary Tree

Problem statement

Maximum number of nodes in any level of the binary tree.

Pattern

BFS (level-order), count nodes at each level.

Solution trick to remember

- Use queue and `q.size()` per level to track width.

Variations

- Width including nulls for complete binary tree
- Width as distance between leftmost and rightmost node indices

C++ code

```
int maxWidth(TreeNode* root) {
    if (!root) return 0;
    int res = 0;
    queue<TreeNode*> q;
    q.push(root);
    while(!q.empty()) {
```

```

        int sz = q.size();
        res = max(res, sz);
        for(int i=0; i<sz; i++){
            TreeNode* node = q.front(); q.pop();
            if(node->left) q.push(node->left);
            if(node->right) q.push(node->right);
        }
    }
    return res;
}

```

Time: $O(n)$

Space: $O(w)$ — max width

6) Count Total Nodes in a Complete Binary Tree

Problem statement

Count nodes in a complete binary tree **efficiently**.

Pattern

- If perfect tree $\rightarrow$ count = $2^h - 1$
- Otherwise $\rightarrow$ recurse

Solution trick to remember

- Compare leftmost height and rightmost height:
 - If equal $\rightarrow$ perfect $\rightarrow$ return $2^h - 1$
 - Else $\rightarrow$ recurse left & right

C++ code

```

int countNodes(TreeNode* root) {
    if(!root) return 0;
    TreeNode* l = root, *r = root;
    int hl=0, hr=0;
    while(l){ hl++; l=l->left;}
    while(r){ hr++; r=r->right;}
    if(hl==hr) return (1<<hl)-1;
    return 1 + countNodes(root->left) + countNodes(root->right);
}

```

Time: $O(\log^2 n)$

Space: $O(h)$

7) Check for Children Sum Property

Problem statement

Check if **node value = sum of left & right child values** for every node.

Pattern

DFS post-order.

Solution trick to remember

- Leaf nodes automatically satisfy property
- For non-leaf: check current + recurse left & right

C++ code

```
bool isChildrenSum(TreeNode* root) {
    if(!root || (!root->left && !root->right)) return true;
    int leftVal = root->left ? root->left->val : 0;
    int rightVal = root->right ? root->right->val : 0;
    return (root->val == leftVal + rightVal) &&
           isChildrenSum(root->left) &&
           isChildrenSum(root->right);
}
```

Time: O(n)

Space: O(h)

Memory hooks for Tree Properties

Problem	Trick
Height	post-order, max(left,right)+1
Balanced	propagate -1 if unbalanced
Diameter	max(leftHeight+rightHeight)
Max Path Sum	max(0,left,right)+node, update global
Max Width	BFS, track size per level
Count Nodes (Complete)	Compare leftmost & rightmost height
Children Sum	post-order, leaf auto-pass, check sum

1) Boundary Traversal of Binary Tree

Problem statement

Return the boundary nodes of a binary tree in **anti-clockwise direction**:

1. Root
2. Left boundary (excluding leaves)
3. Leaf nodes (left $\rightarrow$ right)
4. Right boundary (excluding leaves, bottom $\rightarrow$ top)

Pattern

DFS / separate recursive traversals for left boundary, leaves, and right boundary.

Solution trick to remember

- Separate logic for left boundary, right boundary, and leaves.
- Avoid duplicate leaf nodes.
- Traverse right boundary in reverse for correct order.

C++ code

```
void addLeft(TreeNode* node, vector<int>& res) {
    while(node) {
        if(node->left || node->right) res.push_back(node->val);
        node = node->left ? node->left : node->right;
    }
}

void addRight(TreeNode* node, vector<int>& res) {
    vector<int> tmp;
    while(node) {
        if(node->left || node->right) tmp.push_back(node->val);
        node = node->right ? node->right : node->left;
    }
    reverse(tmp.begin(), tmp.end());
    res.insert(res.end(), tmp.begin(), tmp.end());
}

void addLeaves(TreeNode* node, vector<int>& res) {
    if(!node) return;
    if(!node->left && !node->right) { res.push_back(node->val); return; }
    addLeaves(node->left, res);
    addLeaves(node->right, res);
}

vector<int> boundaryTraversal(TreeNode* root) {
    vector<int> res;
    if(!root) return res;
    res.push_back(root->val);
    addLeft(root->left, res);
    addLeaves(root->left, res);
```

```

        addLeaves(root->right, res);
        addRight(root->right, res);
        return res;
    }

```

Time: $O(n)$

Space: $O(h)$ recursion stack

2) Vertical Order Traversal

Problem statement

Return nodes column-wise (left $\rightarrow$ right). For same column, nodes appear top $\rightarrow$ bottom.

Pattern

BFS + map (column $\rightarrow$ vector). Track horizontal distance (HD) from root.

Solution trick to remember

- Root HD = 0
- Left child HD = parentHD - 1
- Right child HD = parentHD + 1
- Use `map<int, vector<int>>` to maintain sorted column order.

C++ code

```

vector<vector<int>>> verticalOrder(TreeNode* root) {
    vector<vector<int>>> res;
    if(!root) return res;
    map<int, vector<int>>> mp;
    queue<pair<TreeNode*, int>> q;
    q.push({root, 0});
    while(!q.empty()){
        auto [node, hd] = q.front(); q.pop();
        mp[hd].push_back(node->val);
        if(node->left) q.push({node->left, hd-1});
        if(node->right) q.push({node->right, hd+1});
    }
    for(auto &[_ , v]: mp) res.push_back(v);
    return res;
}

```

Time: $O(n \log n)$ (map)

Space: $O(n)$

3) Top View of Binary Tree

Problem statement

Return the nodes visible from the **top**. If multiple nodes in same vertical, choose first from top.

Pattern

BFS + map (column $\rightarrow$ first node at that HD).

Solution trick to remember

- Similar to vertical order.
- Only add node to map if HD not yet seen.
- BFS ensures topmost node processed first.

C++ code

```
vector<int> topView(TreeNode* root) {
    vector<int> res;
    if(!root) return res;
    map<int,int> mp; // HD -> first node
    queue<pair<TreeNode*, int>> q;
    q.push({root,0});
    while(!q.empty()){
        auto [node, hd] = q.front(); q.pop();
        if(mp.find(hd) == mp.end()) mp[hd] = node->val;
        if(node->left) q.push({node->left, hd-1});
        if(node->right) q.push({node->right, hd+1});
    }
    for(auto &[_ , val]: mp) res.push_back(val);
    return res;
}
```

Time: $O(n \log n)$

Space: $O(n)$

4) Bottom View of Binary Tree

Problem statement

Return nodes visible from the **bottom**. If multiple nodes in same vertical, choose last from top $\rightarrow$ bottom.

Pattern

BFS + map (column $\rightarrow$ overwrite node at HD).

Solution trick to remember

- Similar to top view, but overwrite every time a node appears at HD.
- BFS ensures bottommost node remains last.

C++ code

```
vector<int> bottomView(TreeNode* root) {
    vector<int> res;
    if(!root) return res;
    map<int,int> mp;
    queue<pair<TreeNode*, int>> q;
    q.push({root,0});
    while(!q.empty()){
        auto [node, hd] = q.front(); q.pop();
        mp[hd] = node->val; // overwrite
        if(node->left) q.push({node->left, hd-1});
        if(node->right) q.push({node->right, hd+1});
    }
    for(auto &[_ , val]: mp) res.push_back(val);
    return res;
}
```

Time: $O(n \log n)$

Space: $O(n)$

5) Right / Left View of Binary Tree

Problem statement

Return nodes visible from the **right** (or left) side of tree.

Pattern

BFS level-order, take first (left view) or last (right view) node of each level.

Solution trick to remember

- BFS level traversal $\rightarrow$ pick leftmost or rightmost per level.

C++ code

```
vector<int> rightView(TreeNode* root) {
    vector<int> res;
    if(!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while(!q.empty()){
        int sz = q.size();
        for(int i=0;i<sz;i++){
            TreeNode* node = q.front(); q.pop();
            if(i==sz-1) res.push_back(node->val); // rightmost
        }
    }
    return res;
}
```

```

        if(node->left) q.push(node->left);
        if(node->right) q.push(node->right);
    }
}
return res;
}

vector<int> leftView(TreeNode* root) {
    vector<int> res;
    if(!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while(!q.empty()){
        int sz = q.size();
        for(int i=0;i<sz;i++){
            TreeNode* node = q.front(); q.pop();
            if(i==0) res.push_back(node->val); // leftmost
            if(node->left) q.push(node->left);
            if(node->right) q.push(node->right);
        }
    }
    return res;
}

```

Time: $O(n)$

Space: $O(w)$ — max width per level



Memory Hooks for Tree Views

Problem

Trick

Boundary Left boundary $\rightarrow$ leaves $\rightarrow$ right boundary reversed

Vertical BFS + HD $\rightarrow$ map<HD, vector>

Top BFS + map, insert only first HD

Bottom BFS + map, overwrite each HD

Right / Left BFS level-order, pick last/first node per level

1) Check if Two Trees are Identical

Problem statement

Given roots p and q , check if the two binary trees are structurally identical and have the same node values.

Pattern

DFS (post-order or pre-order) on both trees simultaneously.

Trick to remember

Three checks at each node:

- both null ✓
- one null ✗
- values equal & children equal ✓d

Variations

- Check “same structure” ignoring values.
- Check if one is a subtree of another (use identical as a subroutine).

C++ (recursive, clean & optimal)

```
#include <bits/stdc++.h>
using namespace std;

bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q) return false;
    if (p->val != q->val) return false;
    return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}
```

Time: O(n) where n = min(nodes in p, q)

Space: O(h) recursion stack

2) Symmetric Binary Tree

Problem statement

Given root, check if the tree is a mirror of itself around its center.

Pattern

Mirror-check recursion between `left` and `right` subtrees.

Trick to remember

Pairs must match:

- `L->val == R->val`
- mirror children: `L->left ↔ R->right` and `L->right ↔ R->left`

Variations

- Iterative solution using a queue of pairs.

C++ (recursive)

```
bool isMirror(TreeNode* a, TreeNode* b) {
    if (!a && !b) return true;
    if (!a || !b) return false;
    return a->val == b->val
        && isMirror(a->left, b->right)
        && isMirror(a->right, b->left);
}

bool isSymmetric(TreeNode* root) {
    if (!root) return true;
    return isMirror(root->left, root->right);
}
```

Time: $O(n)$

Space: $O(h)$

3) Root to Node Path in Binary Tree

Problem statement

Given `root` and a target value `x`, return the path (values) from root to that node. If not found, return empty.

Pattern

DFS with backtracking.

Trick to remember

Push $\rightarrow$ try left/right $\rightarrow$ if not found, pop (undo). Classic backtracking.

Variations

- Path to a leaf with a given sum (root-to-leaf sum).
- Return node pointers instead of values.

C++ (backtracking)

```
bool getPathHelper(TreeNode* root, int x, vector<int>& path) {
    if (!root) return false;
    path.push_back(root->val);
    if (root->val == x) return true;
    if (getPathHelper(root->left, x, path) || getPathHelper(root->right, x,
path))
        return true;
    path.pop_back();
    return false;
}
```

```
vector<int> rootToNodePath(TreeNode* root, int x) {
    vector<int> path;
    getPathHelper(root, x, path);
    return path; // empty if not found
}
```

Time: $O(n)$

Space: $O(h)$

4) Lowest Common Ancestor (LCA) in a Binary Tree

Problem statement

Given `root` and two nodes `p` and `q` (pointers), return their LCA (first common ancestor).

Pattern

Post-order recursion: return a node pointer if found in subtree.

Trick to remember

At node:

- If `node == p` or `q` → return node
- Recurse left & right →
 - if both non-null → current is LCA
 - else return the non-null side

Variations

- If nodes may not exist, add checks.
- Many queries → preprocess with parent + binary lifting.

C++ (clean recursive)

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* L = lowestCommonAncestor(root->left, p, q);
    TreeNode* R = lowestCommonAncestor(root->right, p, q);
    if (L && R) return root;
    return L ? L : R;
}
```

Time: $O(n)$

Space: $O(h)$

5) Print All the Nodes at Distance K (from a target node)

Problem statement

Given `root`, a `target` node pointer, and `k`, return all node values at distance `k` from `target` (any direction).

Pattern

Build **parent map** (`tree` $\rightarrow$ `undirected graph`) + BFS from `target` up to `k` levels.

Trick to remember

Two-phase:

1. BFS to map `parent[node]`
2. BFS from `target`, expanding to `left`, `right`, and `parent` (visited set prevents backtracking)

Variations

- If `target` given as value, first find pointer to that node.
- Return sorted values if needed.

C++ (parent-map + BFS)

```
void buildParent(TreeNode* root, unordered_map<TreeNode*, TreeNode*>& parent)
{
    if (!root) return;
    queue<TreeNode*> q; q.push(root);
    parent[root] = nullptr;
    while (!q.empty()) {
        TreeNode* cur = q.front(); q.pop();
        if (cur->left) { parent[cur->left] = cur; q.push(cur->left); }
        if (cur->right) { parent[cur->right] = cur; q.push(cur->right); }
    }
}

vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
    if (!root || !target) return {};
    unordered_map<TreeNode*, TreeNode*> parent;
    buildParent(root, parent);

    unordered_set<TreeNode*> vis;
    queue<TreeNode*> q;
    q.push(target); vis.insert(target);
    int dist = 0;
```

```

while (!q.empty()) {
    int sz = q.size();
    if (dist == k) { // collect all at this level
        vector<int> out; out.reserve(sz);
        while (sz--) { out.push_back(q.front()->val); q.pop(); }
        return out;
    }
    while (sz--) {
        TreeNode* cur = q.front(); q.pop();
        if (cur->left && !vis.count(cur->left)) { vis.insert(cur->left); q.push(cur->left); }
        if (cur->right && !vis.count(cur->right)) { vis.insert(cur->right); q.push(cur->right); }
        TreeNode* par = parent[cur];
        if (par && !vis.count(par)) { vis.insert(par); q.push(par); }
    }
    dist++;
}
return {};
}

```

Time: O(n) (build parents + BFS)

Space: O(n)

6) Minimum Time Taken to Burn the Tree from a Node

Problem statement

Given `root` and a target value `x` where fire starts, every second fire spreads to parent and children. Return **minimum time** to burn the entire tree.

Pattern

Same as distance-K but we **BFS until exhaustion** starting from target; time increments per level.

Trick to remember

- Build `parent` map and find `target` pointer.
- BFS from target; if we ignited any new node in a layer $\rightarrow$ `time++`.
- Answer = number of BFS layers minus 1 (or tracked incrementally).

Variations

- If duplicate values exist, you'll need the actual target pointer instead of just value.

C++ (find target by value + parent BFS)

```

TreeNode* mapParentsAndFindTarget(TreeNode* root, int x,
                                   unordered_map<TreeNode*, TreeNode*>&
parent) {
    if (!root) return nullptr;
    queue<TreeNode*> q; q.push(root);
    parent[root] = nullptr;
    TreeNode* target = nullptr;
    while (!q.empty()) {
        TreeNode* cur = q.front(); q.pop();
        if (cur->val == x) target = cur;
        if (cur->left) { parent[cur->left] = cur; q.push(cur->left); }
        if (cur->right) { parent[cur->right] = cur; q.push(cur->right); }
    }
    return target;
}

int minTimeToBurn(TreeNode* root, int x) {
    if (!root) return 0;
    unordered_map<TreeNode*, TreeNode*> parent;
    TreeNode* target = mapParentsAndFindTarget(root, x, parent);
    if (!target) return 0; // if not found

    unordered_set<TreeNode*> vis;
    queue<TreeNode*> q; q.push(target); vis.insert(target);
    int time = 0;

    while (!q.empty()) {
        int sz = q.size();
        bool any = false;
        while (sz--) {
            TreeNode* cur = q.front(); q.pop();
            if (cur->left && !vis.count(cur->left)) {
                vis.insert(cur->left); q.push(cur->left); any = true;
            }
            if (cur->right && !vis.count(cur->right)) {
                vis.insert(cur->right); q.push(cur->right); any = true;
            }
            TreeNode* par = parent[cur];
            if (par && !vis.count(par)) {
                vis.insert(par); q.push(par); any = true;
            }
        }
        if (any) time++; // only if fire spread further in this second
    }
    return time;
}

```

Time: $O(n)$

Space: $O(n)$

✓ Quick Memory Hooks

- **Identical** → “both null ✓, one null ✗, else val+children”
- **Symmetric** → mirror pairs: $(L.val == R.val) \ \& \ L.left \leftrightarrow R.right \ \& \ L.right \leftrightarrow R.left$
- **Root** → **Node Path** → push, try left/right, pop if dead-end
- **LCA** → return p/q/self; if both sides return, self is LCA
- **Distance K** → parent map + BFS k layers
- **Burn Time** → same BFS; increment time only if spread happened

1. Requirements Needed to Construct a Unique Binary Tree (Theory)

Problem Statement

Given traversal(s), determine if you can uniquely construct a binary tree.

Pattern

- Inorder + Preorder → Unique Tree
- Inorder + Postorder → Unique Tree
- Preorder + Postorder → ✗ **Not unique** (multiple trees possible).

Trick to Remember

- **Inorder + (Any other order) = Unique Tree**
- Without **Inorder**, ambiguity arises.

Variation

- Sometimes they ask: “*Why can’t we build using Preorder + Postorder?*” → Because multiple tree structures satisfy the same traversal.

2. Construct Binary Tree from Inorder & Preorder

Problem Statement

Given inorder and preorder traversal arrays, construct the binary tree.

Pattern

- Preorder → gives **root**.
- Inorder → splits into **left** and **right subtrees**.

💡 Trick to Remember

1. Take `pre[preIndex++]` as root.
2. Find root in `inorder` → split into left & right.
3. Recurse for left and right.

✅ Variation

- Construct from *Inorder & Postorder* (same method, but post gives root at last).

This is a **tricky but important edge case**:

👉 If `inorder` and `preorder` both contain **duplicates**, then the tree reconstruction is **not unique** anymore.

Why? Because the same value can appear multiple times in `inorder`, so when you pick that value from `preorder` as the root, you won't know which exact occurrence in `inorder` corresponds to it.

◆ Example

```
Preorder = [4, 2, 4, 6]
Inorder  = [2, 4, 4, 6]
```

At the root (`preorder[0] = 4`), `inorder` has **two 4s**:

- If we choose the 4 at index 1 → left = [2], right = [4,6]
- If we choose the 4 at index 2 → left = [2,4], right = [6]

Both are valid → so reconstruction is **ambiguous**.

Conclusion

- **Unique elements** → perfect reconstruction.
- **Duplicates** → **ambiguous** → **multiple trees possible**.
- To handle duplicates:
 - Either disallow them,
 - Or augment nodes with IDs / use extra traversal info,
 - Or enforce a rule (like leftmost index) to get *a* valid tree (not necessarily unique)

Clean C++ Code

```
class Solution {
public:
    int preIndex = 0;
    unordered_map<int, int> inorderMap;

    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        for (int i = 0; i < inorder.size(); i++) inorderMap[inorder[i]] = i;
        return build(preorder, 0, inorder.size()-1);
    }

    TreeNode* build(vector<int>& preorder, int inStart, int inEnd) {
        if (inStart > inEnd) return NULL;

        TreeNode* root = new TreeNode(preorder[preIndex++]);
        int inIndex = inorderMap[root->val];

        root->left = build(preorder, inStart, inIndex - 1);
        root->right = build(preorder, inIndex + 1, inEnd);

        return root;
    }
};
```

- **Time Complexity:** $O(n)$ (using hashmap for inorder lookup).
 - **Space Complexity:** $O(n)$ recursion stack + map.
-

3. Construct Binary Tree from Postorder & Inorder

Problem Statement

Given inorder and postorder traversal arrays, construct the binary tree.

Pattern

- Postorder $\rightarrow$ gives **root** at end.
- Inorder $\rightarrow$ splits into left & right.

Trick to Remember

1. Take `post[postIndex--]` as root.
2. Find root in inorder $\rightarrow$ split into left & right.
3. Build **right first** (because postorder goes Left $\rightarrow$ Right $\rightarrow$ Root).

Clean C++ Code

```
class Solution {
public:
```

```

int postIndex;
unordered_map<int,int> inorderMap;

TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
    postIndex = postorder.size()-1;
    for (int i = 0; i < inorder.size(); i++) inorderMap[inorder[i]] = i;
    return build(postorder, 0, inorder.size()-1);
}

TreeNode* build(vector<int>& postorder, int inStart, int inEnd) {
    if (inStart > inEnd) return NULL;

    TreeNode* root = new TreeNode(postorder[postIndex--]);
    int inIndex = inorderMap[root->val];

    root->right = build(postorder, inIndex+1, inEnd);
    root->left = build(postorder, inStart, inIndex-1);

    return root;
}
};

```

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(n)$

4. Serialize and Deserialize a Binary Tree

Problem Statement

Serialize (convert tree $\rightarrow$ string/array) and Deserialize (string/array $\rightarrow$ tree).

Pattern

- Use BFS (level order) with `null` markers.
- Store nodes in string/array $\rightarrow$ rebuild using queue.

Trick to Remember

- Think of **serialization as saving tree for transport**.
- BFS + NULL ensures full structure preserved.

Clean C++ Code

```

class Codec {
public:
    string serialize(TreeNode* root) {
        if (!root) return "";
        queue<TreeNode*> q;

```

```

q.push(root);
string s = "";

while (!q.empty()) {
    TreeNode* cur = q.front(); q.pop();
    if (!cur) s += "N ";
    else {
        s += to_string(cur->val) + " ";
        q.push(cur->left);
        q.push(cur->right);
    }
}
return s;
}

TreeNode* deserialize(string data) {
    if (data.empty()) return NULL;
    stringstream ss(data);
    string val; ss >> val;
    TreeNode* root = new TreeNode(stoi(val));
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode* node = q.front(); q.pop();

        if (!(ss >> val)) break;
        if (val != "N") {
            node->left = new TreeNode(stoi(val));
            q.push(node->left);
        }

        if (!(ss >> val)) break;
        if (val != "N") {
            node->right = new TreeNode(stoi(val));
            q.push(node->right);
        }
    }
    return root;
}
};

```

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(n)$

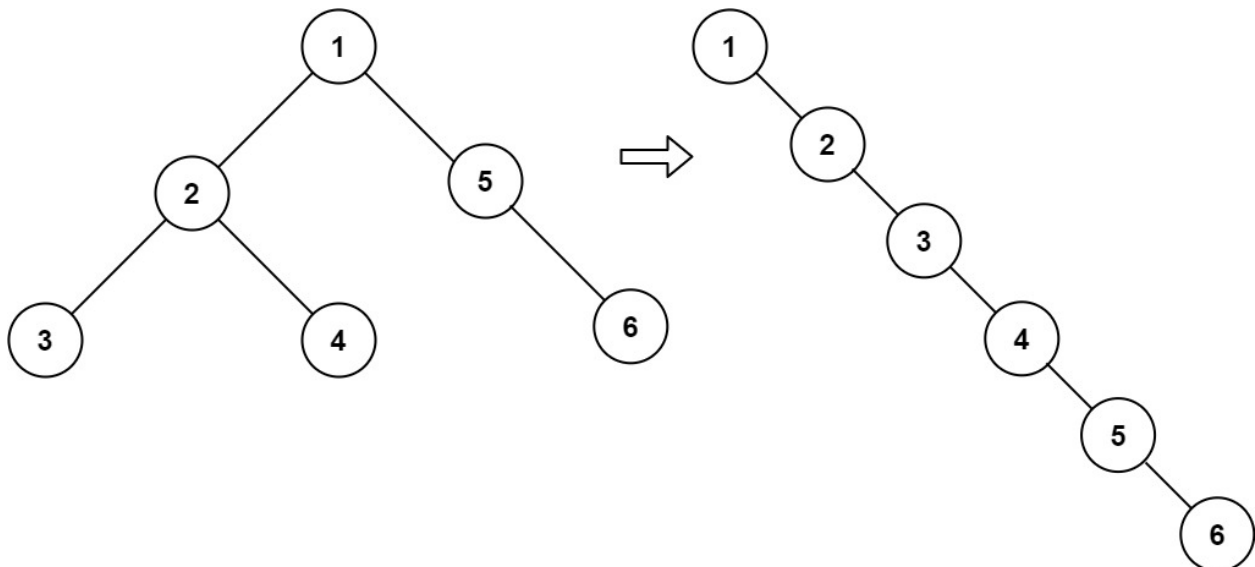
5. Flatten Binary Tree to Linked List

 [Problem Statement](#)

Given the root of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the right child pointer points to the next node in the list and the left child pointer is always null.
- The "linked list" should be in the same order as a [pre-order traversal](#) of the binary tree.

Example 1:



Input: root = [1,2,5,3,4,null,6]

Output: [1,null,2,null,3,null,4,null,5,null,6]

Flatten a binary tree into a linked list in-place (follow preorder).

Pattern

- Use recursion or Morris Traversal.
- Each node's left should become `NULL`, right becomes the next in preorder.

Trick to Remember

- Think: **Preorder** → **Linked List**.
- Recursively flatten left and right, then attach left to right.

Clean C++ Code

```

class Solution {
public:
    void flatten(TreeNode* root) {
        TreeNode* cur = root;
        while (cur) {
            if (cur->left) {
                TreeNode* prev = cur->left;
  
```

```

        while (prev->right) prev = prev->right;
        prev->right = cur->right;
        cur->right = cur->left;
        cur->left = NULL;
    }
    cur = cur->right;
}
};

```

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$ (Morris traversal style).

1. Search in a Binary Search Tree

Problem:

Given a BST root and a value x , check if x exists in the BST.

Pattern:

👉 **BST Traversal (Left < Root < Right).**

Trick:

At each node:

- if $x == \text{node->val} \rightarrow$ found.
- if $x < \text{node->val} \rightarrow$ go **left**.
- if $x > \text{node->val} \rightarrow$ go **right**.

Variations:

- Iterative vs Recursive.
- Return the node instead of bool.
- Count frequency if duplicates allowed.

C++ Code:

```

TreeNode* searchBST(TreeNode* root, int x) {
    while (root && root->val != x) {
        root = (x < root->val) ? root->left : root->right;
    }
    return root; // null if not found
}

```

Complexity: $O(h) \rightarrow O(\log n)$ avg, $O(n)$ worst (skewed).

Space: $O(1)$ iterative, $O(h)$ recursive.

2. Find Min/Max in BST

Problem:

Find the minimum (leftmost) and maximum (rightmost) value in BST.

Pattern:

👉 Leftmost = Min, Rightmost = Max.

Trick:

Keep going left $\rightarrow$ min.

Keep going right $\rightarrow$ max.

Variations:

- Return node instead of value.
- Handle duplicates.

C++ Code:

```
int findMin(TreeNode* root) {
    if (!root) return -1;
    while (root->left) root = root->left;
    return root->val;
}
int findMax(TreeNode* root) {
    if (!root) return -1;
    while (root->right) root = root->right;
    return root->val;
}
```

Complexity: $O(h)$

3. Ceil in BST

Problem:

Find smallest element $\geq x$.

Pattern:

👉 Modified BST traversal.

Trick:

- If $root->val == x \rightarrow$ return $root->val$.
- If $root->val < x \rightarrow$ move right.

- If `root->val > x` → store `ans = root->val` and move left.

Variations:

- Floor problem (mirror logic).
- If no ceil exists → return -1.

C++ Code:

```
int findCeil(TreeNode* root, int x) {
    int ans = -1;
    while (root) {
        if (root->val == x) return x;
        else if (root->val < x) root = root->right;
        else ans = root->val, root = root->left;
    }
    return ans;
}
```

Complexity: $O(h)$

4. Floor in BST

Problem:

Find largest element $\leq x$.

Pattern:

👉 Mirror of Ceil.

Trick:

- If `root->val == x` → return `root->val`.
- If `root->val > x` → move left.
- If `root->val < x` → store `ans = root->val` and move right.

C++ Code:

```
int findFloor(TreeNode* root, int x) {
    int ans = -1;
    while (root) {
        if (root->val == x) return x;
        else if (root->val > x) root = root->left;
        else ans = root->val, root = root->right;
    }
    return ans;
}
```

Complexity: $O(h)$

5. Insert a Node in BST

Problem:

Insert a node with value x into BST.

Pattern:

👉 Binary search till null, then attach.

Trick:

- Traverse as if searching.
- If `root==NULL`, create new node.

C++ Code:

```
TreeNode* insertBST(TreeNode* root, int x) {
    if (!root) return new TreeNode(x);
    if (x < root->val) root->left = insertBST(root->left, x);
    else if (x > root->val) root->right = insertBST(root->right, x);
    return root;
}
```

Complexity: $O(h)$

6. Delete a Node in BST

Problem:

Delete a node with value x from BST.

Pattern:

👉 Three cases:

1. Leaf node $\rightarrow$ delete directly.
2. One child $\rightarrow$ replace with child.
3. Two children $\rightarrow$ replace with **inorder successor** (min in right subtree).

Trick:

Always replace with successor (or predecessor) $\rightarrow$ ensures BST property.

C++ Code:

```
TreeNode* findMinNode(TreeNode* root) {
    while (root && root->left) root = root->left;
    return root;
}

TreeNode* deleteNode(TreeNode* root, int x) {
    if (!root) return NULL;
    if (x < root->val) root->left = deleteNode(root->left, x);
    else if (x > root->val) root->right = deleteNode(root->right, x);
    else {
        if (!root->left) return root->right;
        else if (!root->right) return root->left;
        TreeNode* succ = findMinNode(root->right);
        root->val = succ->val;
        root->right = deleteNode(root->right, succ->val);
    }
    return root;
}
```

Complexity: $O(h)$

✓ CRUD Pattern Summary (Easy to Remember):

- **Search** → **Min/Max** → **Ceil/Floor** → **Insert** → **Delete**
This is just like **Database operations** but on a BST.

◆ 2. BST Property Utilization

1 Check if a Tree is a BST

Problem:

Given a binary tree, check if it is a valid Binary Search Tree (BST).

Pattern:

- Inorder of BST should be strictly increasing.
- Or use range `[min, max]` recursion.

Trick:

Think "**Inorder Sorted = BST**".

If inorder ever violates sorted property → Not BST.

Variations:

- Validate BST using inorder array.
- Validate BST using min/max recursion.

Optimized Code:

```
class Solution {
public:
    bool isValidBST(TreeNode* root, long long low = LLONG_MIN, long long high = LLONG_MAX) {
        if (!root) return true;
        if (root->val <= low || root->val >= high) return false;
        return isValidBST(root->left, low, root->val) &&
            isValidBST(root->right, root->val, high);
    }
};
```

Complexity:

- Time: $O(N)$
 - Space: $O(H)$ recursion stack
-

2 LCA in BST

Problem:

Find the **Lowest Common Ancestor (LCA)** of two nodes in a BST.

Pattern:

- Use BST property:
 - If both $p, q < \text{root}$ → go left.
 - If both $p, q > \text{root}$ → go right.
 - Else root is LCA.

Trick:

"Split point" in BST is always the LCA.

Variation:

- LCA in Binary Tree (no BST property → full recursion).

Optimized Code:

```
class Solution {
public:
```

```

TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q)
{
    while (root) {
        if (p->val < root->val && q->val < root->val) root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root; // Split point
    }
    return nullptr;
};

```

Complexity:

- Time: **O(H)**
- Space: **O(1)**

3 Inorder Successor / Predecessor

Problem:

Find the next (successor) or previous (predecessor) node of a given key in BST.

Pattern:

- Inorder = Sorted → Successor = next bigger, Predecessor = next smaller.
- Use BST traversal with candidate update.

Trick:

Think like **binary search in tree**:

- Successor → go right but keep smaller candidate.
- Predecessor → go left but keep larger candidate.

Variation:

- Successor/Predecessor of all nodes in one traversal.

Optimized Code:

```

class Solution {
public:
    TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {
        TreeNode* succ = nullptr;
        while (root) {
            if (p->val < root->val) {
                succ = root; // candidate
                root = root->left;
            }
            else root = root->right;
        }
        return succ;
    }
};

```

```

        } else {
            root = root->right;
        }
    }
    return succ;
}

TreeNode* inorderPredecessor(TreeNode* root, TreeNode* p) {
    TreeNode* pred = nullptr;
    while (root) {
        if (p->val > root->val) {
            pred = root; // candidate
            root = root->right;
        } else {
            root = root->left;
        }
    }
    return pred;
}
};

```

Complexity:

- Time: **O(H)**
- Space: **O(1)**

4 K-th Smallest / Largest in BST

Problem:

Find the K-th smallest or largest element in BST.

Pattern:

- Inorder $\rightarrow$ Sorted order.
- K-th smallest = Inorder traversal, count nodes.
- K-th largest = Reverse Inorder.

Trick:

Think "Sorted Array but Tree traversal".

Do **inorder traversal with counter**.

Variation:

- Augmented BST (store subtree sizes for $O(\log N)$ queries).
- Multiple queries $\rightarrow$ use segment tree / order statistic tree.

Optimized Code:

```

class Solution {
public:
    int cnt = 0, ans = -1;
    void inorder(TreeNode* root, int k) {
        if (!root || ans != -1) return;
        inorder(root->left, k);
        cnt++;
        if (cnt == k) { ans = root->val; return; }
        inorder(root->right, k);
    }

    int kthSmallest(TreeNode* root, int k) {
        inorder(root, k);
        return ans;
    }

    int kthLargest(TreeNode* root, int k) {
        cnt = 0, ans = -1;
        reverseInorder(root, k);
        return ans;
    }

    void reverseInorder(TreeNode* root, int k) {
        if (!root || ans != -1) return;
        reverseInorder(root->right, k);
        cnt++;
        if (cnt == k) { ans = root->val; return; }
        reverseInorder(root->left, k);
    }
};

```

Complexity:

- Time: $O(H + K)$
- Space: $O(H)$

◆ 3. BST Construction Problems

3.1 Construct a BST from Preorder Traversal

Problem Statement

You are given the preorder traversal of a BST. Construct the BST and return its root.

Pattern

- Preorder = (root → left → right)

- For BST, elements are inserted in order, respecting bounds.
- Use recursion with **lower & upper bounds** or iterative stack.

 Trick to Remember

Think “**Preorder means I already know the root, next values must fall in [min, root] for left, and [root, max] for right**”.

 Variations

- Construct BST from:
 - Preorder + Inorder
 - Postorder
 - Level Order

 Optimized Code (Using Bounds, O(n))

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node* left;
    Node* right;
    Node(int x): val(x), left(NULL), right(NULL) {}
};

Node* buildBST(vector<int>& preorder, int& i, int bound) {
    if(i == preorder.size() || preorder[i] > bound) return NULL;
    Node* root = new Node(preorder[i++]);
    root->left = buildBST(preorder, i, root->val);
    root->right = buildBST(preorder, i, bound);
    return root;
}

Node* bstFromPreorder(vector<int>& preorder) {
    int i = 0;
    return buildBST(preorder, i, INT_MAX);
}
```

 Complexity

- **Time:** O(n)
- **Space:** O(h) recursion stack (O(n) worst, O(log n) best balanced tree).

3.2 Merge 2 BSTs

Problem Statement

You are given 2 BSTs. Merge them into a single balanced BST.

Pattern

1. Get inorder traversal of both BSTs (sorted arrays).
2. Merge arrays like merge-sort.
3. Build balanced BST from merged array.

Trick to Remember

- **Inorder = sorted list.**
- Merge like **merging 2 sorted LinkedLists**.
- Build balanced BST = same as converting sorted array $\rightarrow$ BST.

Variations

- Merge k BSTs (use k-way merge with heap).
- Merge BST + DLL (flattened tree).

Optimized Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node* left;
    Node* right;
    Node(int x): val(x), left(NULL), right(NULL) {}
};

// Step 1: Inorder traversal
void inorder(Node* root, vector<int>& arr) {
    if(!root) return;
    inorder(root->left, arr);
    arr.push_back(root->val);
    inorder(root->right, arr);
}

// Step 2: Merge two sorted arrays
vector<int> mergeArrays(vector<int>& a, vector<int>& b) {
    vector<int> res;
    int i = 0, j = 0;
    while(i < a.size() && j < b.size()) {
        if(a[i] < b[j]) res.push_back(a[i++]);
        else res.push_back(b[j++]);
    }
    while(i < a.size()) res.push_back(a[i++]);
    while(j < b.size()) res.push_back(b[j++]);
}
```

```

        return res;
    }

    // Step 3: Build balanced BST
    Node* buildBalancedBST(vector<int>& arr, int l, int r) {
        if(l > r) return NULL;
        int mid = (l + r) / 2;
        Node* root = new Node(arr[mid]);
        root->left = buildBalancedBST(arr, l, mid - 1);
        root->right = buildBalancedBST(arr, mid + 1, r);
        return root;
    }

    Node* mergeBSTs(Node* root1, Node* root2) {
        vector<int> a, b;
        inorder(root1, a);
        inorder(root2, b);
        vector<int> merged = mergeArrays(a, b);
        return buildBalancedBST(merged, 0, merged.size() - 1);
    }

```

Complexity

- **Inorder Traversal:** $O(n1 + n2)$
- **Merging Arrays:** $O(n1 + n2)$
- **Building Balanced BST:** $O(n1 + n2)$
- ☒ **Overall:** $O(n1 + n2)$
- **Space:** $O(n1 + n2)$



Master Trick for Construction Problems

- If given **traversal** → Use **bounds** or simulate inorder.
- If given **multiple BSTs** → Convert to arrays (sorted), merge, then build BST.
- Always think: **Inorder = Sorted Array** → merge/build.



4. Advanced / Tricky Problems in BST

1. Two Sum in BST

Problem Statement

Given a BST and an integer k , check if there exist two nodes whose sum is equal to k .

Pattern

- Convert BST to **sorted inorder array** → Use **2 pointers** like “Two Sum in Sorted Array”.
- Or directly use **HashSet** while doing inorder traversal.

Trick to Remember

👉 Whenever you see “Two Sum in BST” → **Inorder = sorted array** → use **Two Pointers**.
If space is tight, do inorder with **stack + set**.

Variations

- Return true/false if pair exists.
- Return the pair(s).
- Count number of pairs.

Optimized Code (C++ | O(N) time, O(N) space)

```
class Solution {
public:
    void inorder(TreeNode* root, vector<int>& arr) {
        if(!root) return;
        inorder(root->left, arr);
        arr.push_back(root->val);
        inorder(root->right, arr);
    }

    bool findTarget(TreeNode* root, int k) {
        vector<int> arr;
        inorder(root, arr); // sorted inorder
        int i = 0, j = arr.size()-1;
        while(i < j) {
            int sum = arr[i] + arr[j];
            if(sum == k) return true;
            else if(sum < k) i++;
            else j--;
        }
        return false;
    }
};
```

Time: O(N), **Space:** O(N)

2. Recover BST

Problem Statement

In a BST, exactly two nodes are swapped by mistake. Restore the BST without changing its structure.

Pattern

- BST inorder should be **sorted**.
- If 2 nodes are swapped, inorder will show **two anomalies**:
 - First anomaly → first element to swap.
 - Second anomaly → second element to swap.
- Swap them back.

Trick to Remember

👉 Think: “If inorder is not sorted → find misplaced nodes → swap them.”
At most 2 wrong nodes.

Variations

- Correct tree in-place.
- Just print which two nodes are swapped.

Optimized Code (C++ | $O(N)$ time, $O(H)$ space recursion stack)

```
class Solution {
public:
    TreeNode* first = NULL;
    TreeNode* second = NULL;
    TreeNode* prev = new TreeNode(INT_MIN);

    void inorder(TreeNode* root) {
        if(!root) return;
        inorder(root->left);

        if(prev && root->val < prev->val) {
            if(!first) first = prev;
            second = root;
        }
        prev = root;

        inorder(root->right);
    }

    void recoverTree(TreeNode* root) {
        inorder(root);
        swap(first->val, second->val);
    }
};
```

Time: $O(N)$, **Space:** $O(H)$ recursion stack.

👉 Can be reduced to **$O(1)$ space** using **Morris Traversal**.

3. Largest BST in Binary Tree

Problem Statement

Given a binary tree (not necessarily BST), find the size of the **largest BST subtree** inside it.

Pattern

- For each node, check if its subtree is BST:
 - Needs **min**, **max**, **size** of left/right subtrees.
- Use **postorder traversal** (bottom-up).
- Return tuple: {isBST, minVal, maxVal, size}.

Trick to Remember

👉 Think “Not every BT is a BST → Need to validate each subtree as BST.”

Return combined info from children.

Variations

- Return **size** of largest BST (most common).
- Return the **root node** of largest BST.

Optimized Code (C++ | $O(N)$ time, $O(H)$ space)

```
struct NodeInfo {
    bool isBST;
    int size, minVal, maxVal;
    NodeInfo(bool b, int s, int mn, int mx): isBST(b), size(s), minVal(mn),
maxVal(mx) {}
};

class Solution {
public:
    int maxSize = 0;

    NodeInfo helper(TreeNode* root) {
        if(!root) return NodeInfo(true, 0, INT_MAX, INT_MIN);

        auto left = helper(root->left);
        auto right = helper(root->right);

        if(left.isBST && right.isBST && root->val > left.maxVal && root->val
< right.minVal) {
```

```

        int sz = left.size + right.size + 1;
        maxSize = max(maxSize, sz);
        return NodeInfo(true, sz, min(root->val, left.minVal), max(root->val, right.maxVal));
    }

    return NodeInfo(false, 0, 0, 0);
}

int largestBSTSubtree(TreeNode* root) {
    helper(root);
    return maxSize;
}
};

```

Time: $O(N)$, **Space:** $O(H)$.

Quick Recap Tricks

- **Two Sum BST** → Inorder sorted + 2 pointers.
- **Recover BST** → Inorder anomalies → swap nodes.
- **Largest BST in BT** → Postorder + return {isBST, min, max, size}